ことばを当てる機械

言語モデル ― 文豪テストから始める

rkskmt

文豪テスト

これから4つの文章を見せる。**本物の夏目漱石**（『吾輩は猫である』）と、**機械が書いた偽物**が混ざっている。どれが本物か、投票してもらう。

ⓘ A

何も顔のまずい例に特に吾輩を出さなくっても、よさそうなものだ。吾輩だって喜多床へ行って顔さえ剃って貰やあ、そんなに人間と異ったところはありゃしない。

ⓘ B

元来主人のように関する主人はどうか心までもあるとも、この気概も何か云う訳にはなし、もう少し自然たるべきものはなるごとく、主人の事件はことには、せられても、たまらんよりとも全くすると知って来ない。

文豪テスト（つづき）

ⓘ C

迷亭はとぼけた顔をして「君のような親切な夫を持った妻君は実に仕合せだな」と独り言のようにいう。障子の蔭でエヘンと云う細君の咳払いが聞える。

ⓘ D

まあどうですか、全く君の事じゃありませんか、知らんと云われているとボイはこの際になれんではない。それではないかと思ってその実はどの人間のようにかかるるんだ。

- 挙手で投票：本物はどれ？（複数可）

何を手がかりに見分けた？



いま挙げた手の**根拠**を言語化しておく。

0. 4つの文章に「本物っぽさ」を◎〇△×で採点
1. いちばん怪しい文章の、怪しい**箇所**に下線 — 単語が変？　つながりが変？　意味が迷子？
2. 下線の理由を一言メモして、隣の人と見比べ。同じ箇所を挙げていたら、そこが「機械のしっぽ」

> **ⓘ ノート**
>
> 採点表は**捨てずに取っておく** — 最終回の種明かしのとき、自分の目がどこまで確かだったか答え合わせできる。

- いま使った「日本語らしさ」の感覚 — **それを数で作り直すのが、この講義の仕事**

答えは最終回に

- 正解の発表は**最終回**。ひとつだけ予告する：

> **ⓘ 約束**
>
> 偽物を書いた機械を、この講義で**ゼロから全部品自作**する。 使うのは、君たちがすでに持っている道具と、新しい概念**たった2つ**。 最終回、君たちは自分のマシンで偽漱石を印刷して帰る。

- 前のコースの約束が「手書き数字を読む機械」だったのと同じ形 — 今度は**書く機械**

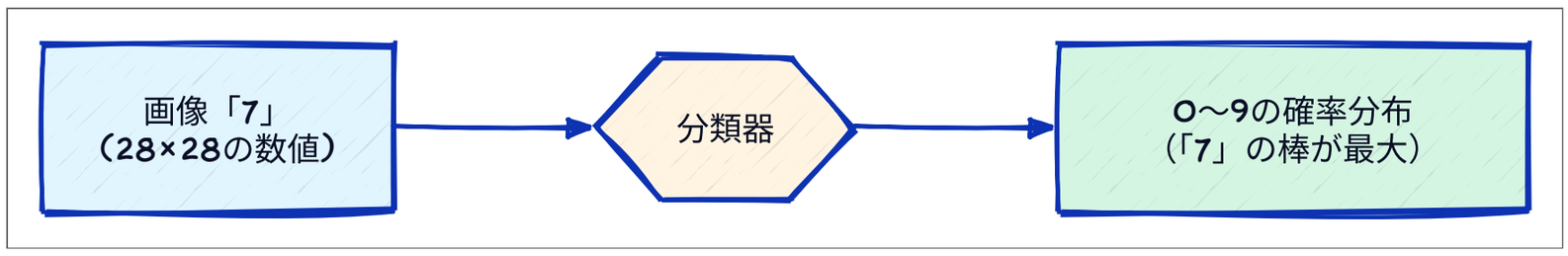
ChatGPT は何をしている機械か

> **ⓘ クイズ**
>
> ChatGPT の中身がやっている仕事を、一言で言うと？ 「考えている」「検索している」「会話している」……？

- 答え：**次に来る言葉を予測している**。それを繰り返しているだけ
- この「次を当てる機械」を**言語モデル**（**language model**）と呼ぶ
- 大規模にしたものが **LLM**（**Large Language Model**）— ChatGPT の本体

前コースのゴールを30秒で

- 前コースの最終回に作ったのは「手書き数字を読む機械」— 種類でいうと**分類器**（classifier）
- 入力は画像（28×28個の数値）。出力は **0〜9 の「棒グラフ」10本**
  → 棒の高さ＝「その数字だと思う度合い」。全部足すと1 — この形を**確率分布**（probability distribution）と呼ぶ
- 「これを入れたらこの棒が高くなるように」と、中の数値を**学習**で調整した



- 採点係は**交差エントロピー**（cross-entropy）という損失 — ひとまず「正解に低い確率を付けるほど重くなる罰点」とだけ。中身は「学習で当てる」の回でもう一度開ける

どこかで見た形 — MNIST との再会

	手書き数字（前コースの最終回）	言語モデル
入力	画像（28×28の数値）	ここまでの文章
出力	**10本**の棒（確率分布）	**語彙全体**ぶんの棒（確率分布）
意味	「どの数字か」	「次の1文字は何か」
損失	交差エントロピー	**交差エントロピー**

- 言語モデルは「次の1文字を当てる**分類器**」でしかない
- 勾配降下・backprop・過学習・PyTorch の3拍子 — **前コースの道具がそのまま全部使える**

> ⓘ **ノート**
>
> 本当に新しい概念は2つだけ：**埋め込み**（ことばを数にする）と**注意**（どこを見るか）。 それ以外は全部、知っている部品の組み替えで済む。

体験：君も言語モデル

ペアで交互に「次の1文字」を予想する。出題者は答えだけ知っている。

- 第1問 「こんにち（　）」
- 第2問 「吾輩は猫であ（　）」
- 第3問 「今日の晩ご飯は（　）」
- 第1問・第2問はほぼ全員正解。第3問は**原理的に当てられない** — この差はどこから来る？

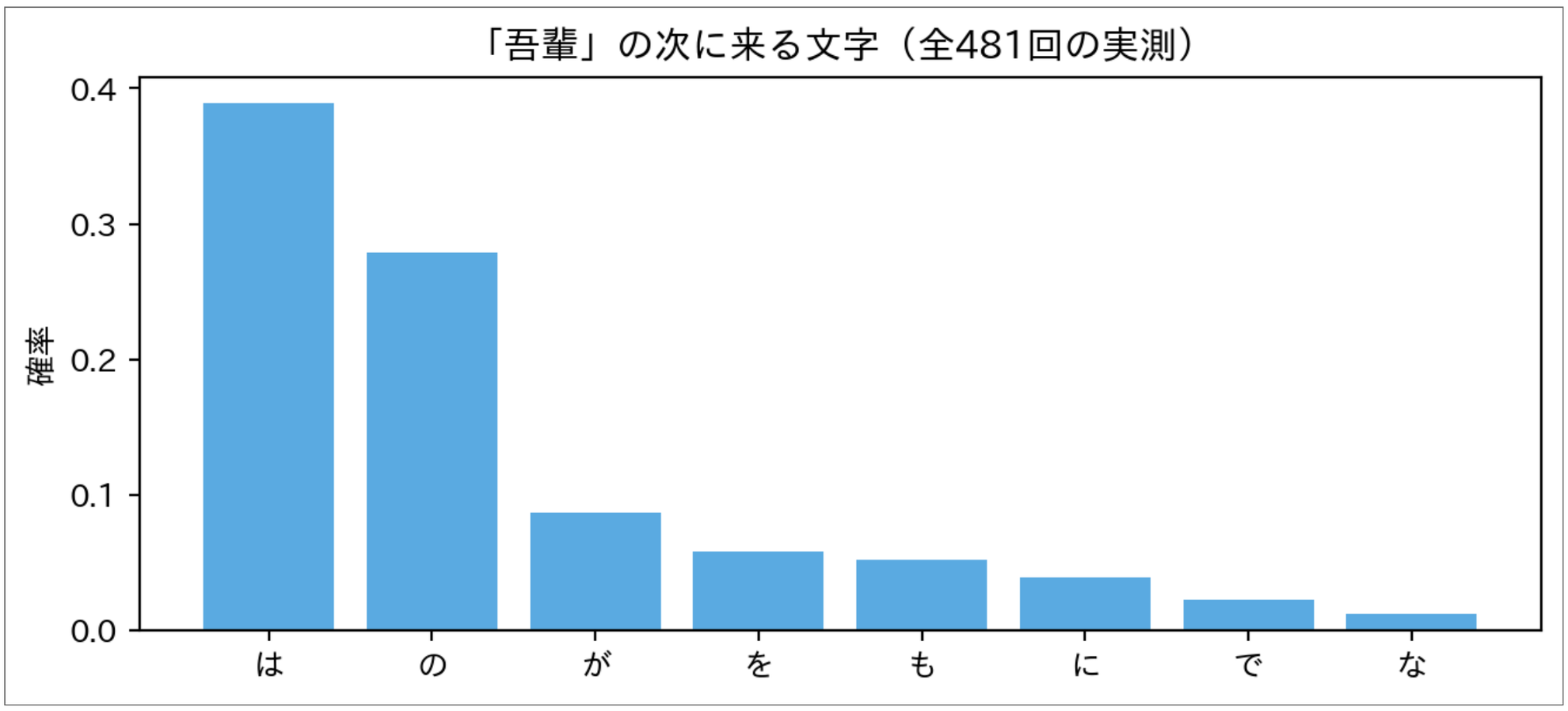
当てやすさには濃淡がある

- 第1問「こんにち（　）」：候補が事実上1つ — **ほぼ確信**
- 第3問「今日の晩ご飯は（　）」：カレーも寿司も鍋もあり — **お手上げの総当たり**
- つまり「次の1文字」には**当てやすさの濃淡**がある — これを数で表したものが**確率分布**
  - → 確信 → 1本の棒がそびえる**尖った**分布
  - → お手上げ → 低い棒がずらりと並ぶ**平らな**分布
- 人間は文脈からこの分布を頭の中で作っている。同じことを機械にやらせたい

「予想」を数に置き換える

① クイズ

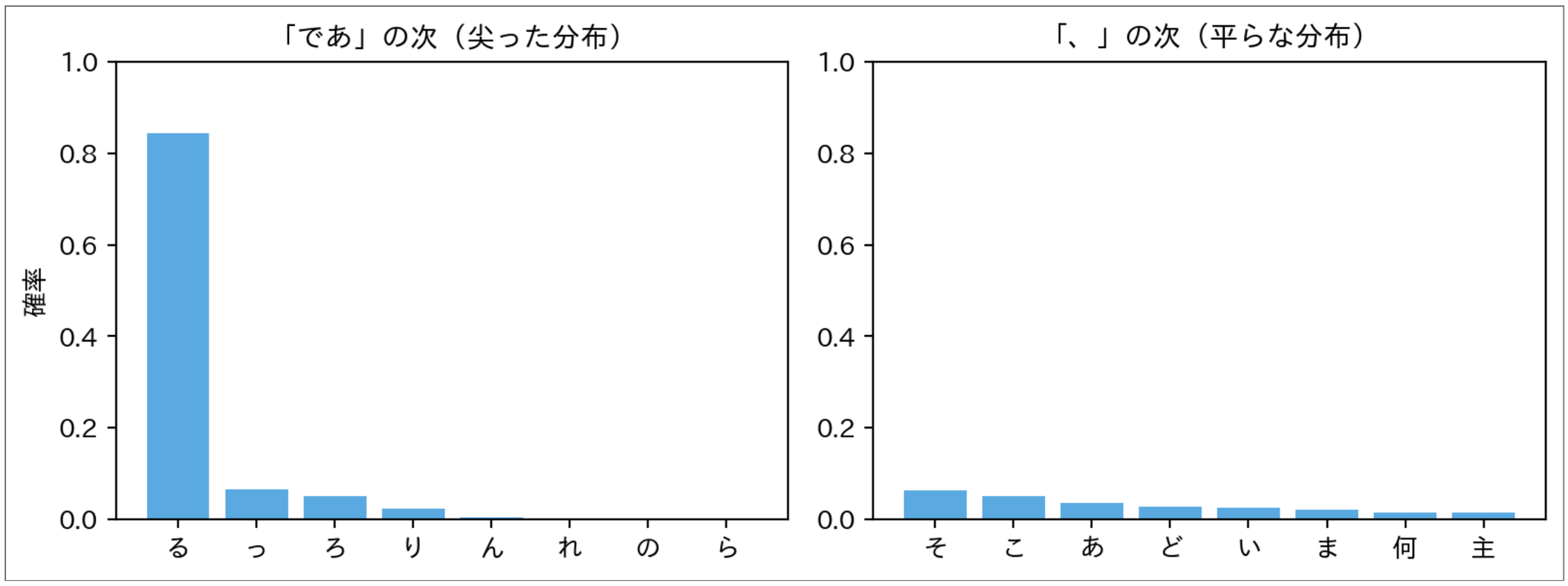
『吾輩は猫である』全文（約32万字）で、「吾輩」の**次**に来た文字を数える。1位はどの文字？2位は？



- 「は」39%、「の」28%…… — 数えただけで「予想」が数になった
- **言語モデルの中身はこの棒グラフの親玉** — どんな文脈にも棒グラフを返す機械

尖った分布と、平らな分布

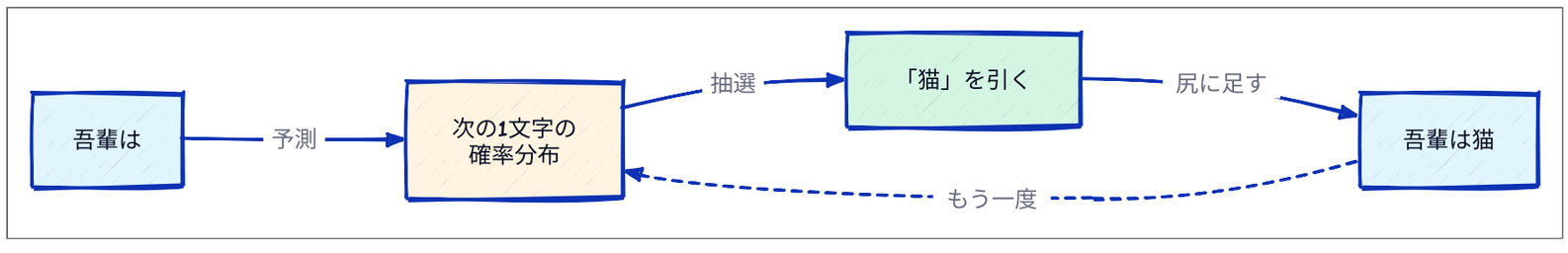
- 体験ゲームの「確信」と「お手上げ」も、数えれば形になる。「であ」の次と「、」の次：



- 「であ」の次は「る」が **84.5%**、候補は全部で**12種類** — 第1問（こんにち）の世界
- 「、」の次は1位の「そ」でも **6.3%**、候補は**1,044種類** — 第3問（晩ご飯）の世界
- 予測のしやすさ＝**分布の形**。言語モデルの実力は、この形をどれだけ本物の続きに近づけられるか

1文字ずつなら、文章も書ける

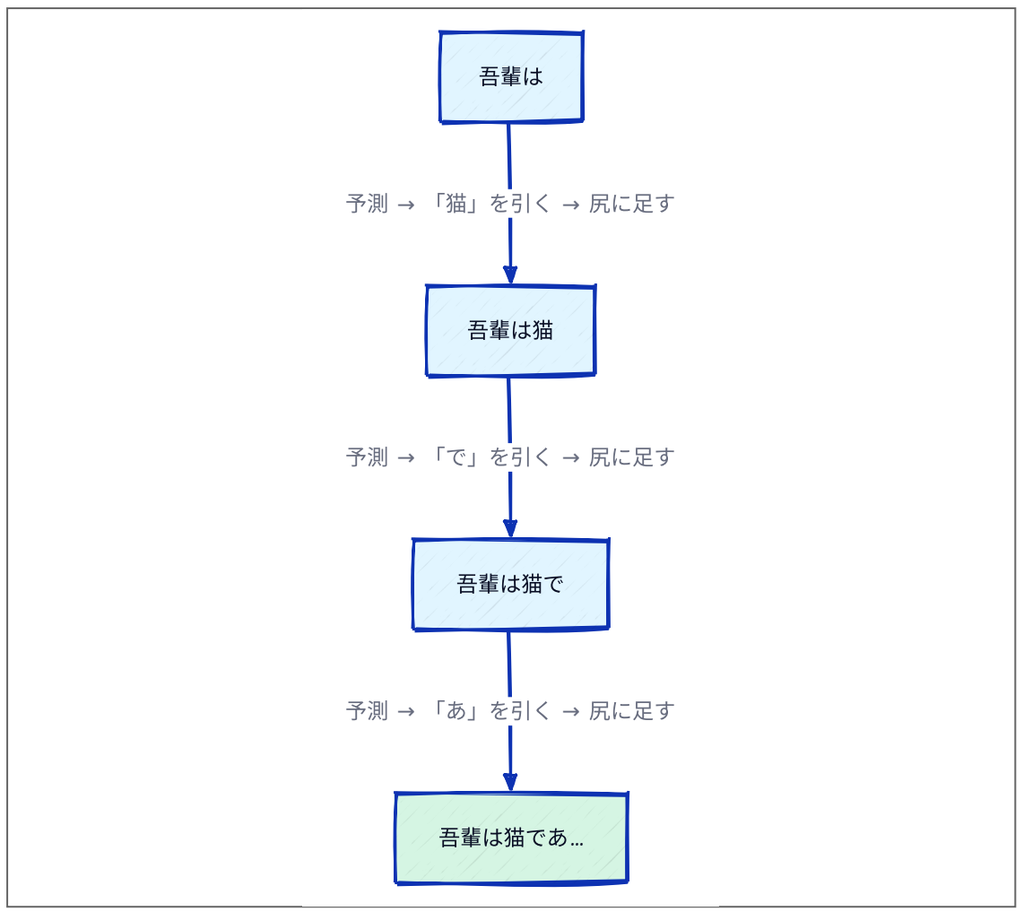
- 予測した1文字を文章の尻に**足して**、また次を予測する — この繰り返し



- **自己回帰**（**autoregressive**）と呼ぶ。ChatGPT が1語ずつ「打っている」ように見えるのは、本当に1語ずつ作っているから
- 「当てる」ができれば「書ける」— だからこの講義は**「次の1文字を当てる」だけを磨き続ける**

生成を1歩ずつほどく

- ぐるぐる回る絵を、まっすぐ伸ばして3歩ぶん：



- 毎歩やることは同じ：いまの文脈を入れる → 棒グラフをもらう → 1本引く → 尻に足す
- 機械は1台きり。**伸びていくのは文脈のほう**
- 「引く」は抽選 — 同じ書き出しでも、実行のたびに違う文章になりうる。ChatGPT の答えが毎回少し違う理由も、ここ（ツマミの調べ方は「生成を操る」の回で）

この講義の道のり

- **架空の地名メーカーを育てる**：数えるだけのモデルから始めて、学習・埋め込み・文脈の窓へ
- **注意（Attention）を開ける**：「どこを見るか」を学習する仕組みを手計算からコードまで
- **GPT を組んで火を入れる**：部品を束ねて『吾輩は猫である』32万字で訓練する
- **推し文豪ボット**：最終回、好きな作家で自分のGPTを育てる — そして文豪テストの種明かし

> ⓘ **ノート**
>
> どの回も「小さく作って動かして、1個だけ賢くする」の繰り返し。 動かないものを3回我慢する回は、ひとつもない。

持ち物の確認

前コースから持ってくる道具（全部使う）：

- **勾配降下法**と学習率 — 学習の背骨
- **交差エントロピー** — 分類の損失。今回も主役
- **backprop / autograd** — `loss.backward()` を信じて使う権利は取得済み
- **過学習と train/val** — 今回も事件を起こす（予告）
- **PyTorch の3拍子** — `zero_grad → backward → step`

> **忘れていても大丈夫**
>
> 名前を見て「なんだっけ」となっても問題ない。どの道具も、**使う回が来たらその場でもう一度説明する**。 この一覧は「これから再会する顔ぶれ」の予告編にすぎない。

演習：数えるだけの予測器

『吾輩は猫である』で「猫」の次に来る文字を数え、確率の高い順に5つ表示せよ。

0. **neko.txt** をダウンロードして、作業フォルダの `data/` に置く
1. `open("data/neko.txt", encoding="utf-8").read()` で全文を読み込み
2. `find` で「猫」の位置を順に探し、その次の1文字を辞書で数える（`collections.Counter` 可）
3. 回数の多い順に5件、「文字：回数」で表示

▸ 解答例を見る

まとめ

- 言語モデル＝**次の1文字を当てる分類器**。MNIST と同じ形、同じ損失
- 予想の濃淡は**確率分布**の形で表せる — 確信は尖り、お手上げは平ら。数えるだけでも分布は作れる（今日の棒グラフ）
- 1文字当てて尻に足す（**自己回帰**）を繰り返せば、文章が書ける
- 新しい概念は**埋め込み**と**注意**の2つだけ — あとは持っている道具の組み替え

現在の設計図　文脈 → **語彙全体の確率** → 次の1文字 → 文脈へ戻す

> **ⓘ 次回予告**
>
> 機械は「あ」という文字をそのままでは食べられない。 まず、ことばを**数**にする — 下ごしらえの回。文豪テストの機械への第一歩。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：尖りと平らを実測する

『吾輩は猫である』で、「であ」の次に来る文字と「、」の次に来る文字をそれぞれ数え、**候補の種類数**と**1位の文字とその確率**を表示せよ。

0. **neko.txt** を **`data/`** に置く（演習と同じ）
1. 演習のコードの「猫」を「であ」「、」に替えて数える（次の文字の位置は **`i + len(文脈)`**）
2. 種類数は **`len(nxt)`**、1位は **`nxt.most_common(1)`** で取り、「種類数」「1位の文字：確率」を表示
3. どちらが「こんにち」型で、どちらが「晩ご飯」型か、本編の図と見比べる

解答例

▸ 解答例を見る

Result

```
「であ」の次: 12種類 | 1位「る」 84.5%
「、」の次: 1044種類 | 1位「そ」 6.3%
```